

Title of the Project here

Project Team

Student 1 19I-1234

Student 2 19I-1234

Student 3 19I-1234

Session 2018-2022

Supervised by

Mr./ Ms./ Dr. Supervisor Name

Co-Supervised by

Mr./ Ms./ Dr. Supervisor Name



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2022

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Client Mobile App	2
1.4	User Classes and Characteristics	4
2	Project Requirements	7
2.1	Use-case/Event Response Table/Storyboarding	7
2.1.1	Detailed Use Cases	9
2.2	Functional Requirements	18
2.2.1	UI/UX and Profile Management	18
2.2.2	Ride Matching and Booking	18
2.2.3	Route Optimization and Navigation	18
2.2.4	Dynamic Pricing Model	18
2.2.5	Safety and Security Features	19
2.2.6	User Feedback and Support	19
2.2.7	Notifications and Alerts	19
2.2.8	User Analytics and Reporting	19
2.2.9	Real-time Ride Analytics and Processing Module	20
2.3	Non-Functional Requirements	20
2.3.1	Reliability	20
2.3.2	Usability	20
2.3.3	Performance	21
2.3.4	Security	21
2.4	Domain Model	21
3	System Overview	23
3.1	Architectural Design	23
3.1.0.1	1. Presentation Layer	23
3.1.0.2	2. Business Logic Layer	23
3.1.0.3	3. Data Layer	23
3.2	Design Models	24

3.2.1	Activity Diagram	24
3.2.2	Data Flow Diagram	30
3.2.2.1	1. User Interaction	32
3.2.2.2	2. Data Input and Processing	32
3.2.2.3	3. Payment Processing	32
3.2.2.4	4. Notification System	33
3.2.2.5	5. Safety and Security	33
3.2.2.6	6. Data Storage	33
3.2.3	System-Level Sequence Diagram	34
3.3	Data Design	35
3.3.0.1	1. User Profiles	37
3.3.0.2	2. Ride Requests	37
3.3.0.3	3. Drivers	38
3.3.0.4	4. Matched Rides	38
3.3.0.5	5. Payments	38
3.3.0.6	6. Ride Reviews	39
3.3.0.7	7. Pricing Analytics	39
3.3.0.8	Data Storage Items:	40
	References	41

List of Figures

2.1	Use Case Diagram	8
2.2	Domain Model Diagram of HumRahi	22
3.1	Architecture Diagram	24
3.2	Activity Diagram for Search Rides	25
3.3	Activity Diagram for View Ride History	26
3.4	Activity Diagram for Make Payment	27
3.5	Activity Diagram for Requesting a Ride	28
3.6	Activity Diagram for Offer a Ride	29
3.7	Activity Diagram for Manage Ride Requests	29
3.8	Activity Diagram for Approve/Verify Users	30
3.9	Data Flow Diagram	31
3.10	System-Level Sequence Diagram	35
3.11	Entity Relationship Diagram	36

List of Tables

2.1	Detailed Use Case for Search Rides	10
2.2	Detailed Use Case for Offer Ride	11
2.3	Detailed Use Case for Make Payment	12
2.4	Detailed Use Case for Manage Profile	13
2.5	Detailed Use Case for Request a Ride	14
2.6	Detailed Use Case for View Ride History	15
2.7	Detailed Use Case for Approve/Verify Users	16
2.8	Detailed Use Case for View Reports	17

Chapter 1

Introduction

The creation of the Humrahi Carpooling Application, a mobile platform designed to offer an affordable, environmentally friendly, and safe option for urban commuting, is presented in this study. Affordable transportation choices are in greater demand as cities become more populated, and worries about growing fuel prices, traffic jams, and environmental damage have also intensified. In order to solve these problems, Humrahi uses sophisticated algorithms for dynamic pricing, real-time safety monitoring, route matching, and customized user experiences. The goal of the software is to improve carpooling for city people by making it more feasible, effective, and accessible.

This paper is designed for academic assessors, including project supervisors, professors, and examination committees who will examine the technical depth, inventiveness, and overall effectiveness of the solution in accomplishing its stated objectives. The study offers a thorough analysis of Humrahi's design, . [?].

1.1 Problem Statement

Urban commuting is becoming increasingly costly due to the under-utilization of private vehicles, contributing to traffic congestion and environmental degradation. Existing carpooling applications face challenges such as limited user adoption, inefficient ride matching, and security concerns, which hinder their effectiveness. Similarly, the current public transportation system is also being crippled by drawbacks like overfilling of passengers, fixed routes, uncomfortable and unsanitary travelling conditions. These issues highlight the need for more efficient and secure carpooling solutions to reduce commuting expenses and mitigate environmental impacts in urban areas, making it a critical issue in the broader context of sustainable urban transportation.

1.2 Scope

The scope of the **Humrahi carpooling application** centers on creating a practical, efficient, and user-friendly ride-sharing platform aimed at urban commuters and university students. The application will feature advanced route-matching algorithms to optimize user convenience and match drivers and passengers based on proximity, destination, and schedule. A dynamic pricing model will ensure fair cost distribution, adjusting based on distance, demand, and timing. To prioritize user safety, Humrahi will implement real-time vehicle tracking and a range of security measures. Recognizing cultural sensitivities, the application will include features specifically supporting conservative female passengers, such as female-only ride options and enhanced privacy settings.

The initial deployment will focus on Islamabad and its adjacent urban areas, where the density of commuters makes carpooling both feasible and impactful. Expansion into rural areas and integration with public transportation are outside the scope, allowing the project to focus on high-density urban regions. The development approach will include a structured launch followed by iterative updates based on user feedback, helping address emerging needs and refining the app's usability. Budget limitations and technological constraints will shape feature prioritization, scalability, and real-time data management. Ethical considerations, especially around data privacy and security, will be paramount, ensuring compliance with relevant regulations and safeguarding user information. By addressing the diverse needs of urban commuters through innovative features, Humrahi seeks to enhance commuting efficiency, reduce transportation costs, and promote sustainable transportation solutions while fostering inclusivity, security, and user comfort.

1.3 Modules

1.3.1 Client Mobile App

1. UI/UX and Profile Management

- Sign-up and login using email, phone number, or social media accounts
- A smooth and accessible user interface catered to user
- User verification and identity checks
- Profile editing and management

2. Ride Matching and Booking

- Ride offer creation by drivers with route details and availability
- Ride request creation by passengers with destination and timing
- Algorithm for optimal ride matching based on proximity, route, and preferences with real time data
- Real-time notifications for ride status updates (e.g., confirmation, cancellations)
- In-app messaging between drivers and passengers

3. Route Optimization and Navigation

- Integration with mapping services for route planning and navigation
- Real-time traffic updates and route adjustments
- Estimated time of arrival (ETA) and distance calculations
- Option for users to view and select preferred routes

4. Dynamic Pricing Model

- Transparent pricing mechanism based on distance, demand, and time
- Fare estimation before ride confirmation
- In-app payment processing with multiple payment options
- Price adjustments for peak times or special conditions

5. Safety and Security Features

- Real-time ride tracking and location sharing
- User ratings and reviews for both drivers and passengers
- Verification of driver licenses and vehicle information

6. User Feedback and Support

- In-app support and helpdesk for user queries and issues
- Feedback forms and rating system for post-ride evaluations

- Utilizes natural language processing (NLP) to automatically analyze user reviews and feedback to detect sentiment, identify common themes, and flag potential issues or complaints
- FAQs and troubleshooting guides

7. Notifications and Alerts

- Push notifications for ride requests, confirmations, and status changes
- Alerts for upcoming rides, cancellations, and delays
- Customizable notification settings for user preferences

8. User Analytics and Reporting

- Data collection on user preferences, ride patterns, and feedback
- Analytics dashboard for monitoring usage and performance
- Reports on environmental impact and cost savings

9. Real-time Ride Analytics and Processing Module

- Integration with Apache Kafka for real-time data streaming and processing during rides
- Collection of live analytics on ride duration, distance, route deviations, and traffic conditions
- Scalable infrastructure to support large volumes of live data for future expansion

1.4 User Classes and Characteristics

User class	Description
Commuters	Commuters represent the primary users of the Humrahi carpooling app, consisting of urban dwellers who regularly travel to workplaces or educational institutions. This group includes university students and office workers who seek an affordable and efficient transportation solution. These users prioritize convenience, cost savings, and safety. A majority of them are expected to book rides daily, using the app during peak commuting hours.
Drivers	Drivers are individuals who own vehicles and are willing to share their ride with other commuters traveling along similar routes. This class includes private vehicle owners, university students with vehicles, and working professionals who want to offset their travel costs by carpooling. Drivers need to interact with features like ride offer creation, route selection, and real-time notifications, and they'll be able to view commuter profiles and ratings for safety.
Female Commuters	This class specifically targets female users who seek carpooling options tailored to safety and privacy concerns. Female commuters may include students and working women who prioritize female-only rides. The app offers unique features, like female-only ride options and enhanced privacy settings, designed to make commuting safer and more comfortable for this group.
Adminsitrators	Administrators are responsible for managing the carpooling application's functionality and safety compliance. Their primary tasks include overseeing user accounts and verification processes, monitoring ride requests, and ensuring adherence to safety protocols. They moderate content to prevent inappropriate behavior, analyze user data to inform strategic decisions, and provide support for user queries and complaints. Additionally, they generate reports on user engagement and safety incidents to ensure transparency and accountability. By centralizing these responsibilities, App Administrators play a crucial role in maintaining an efficient and secure ride-sharing platform.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

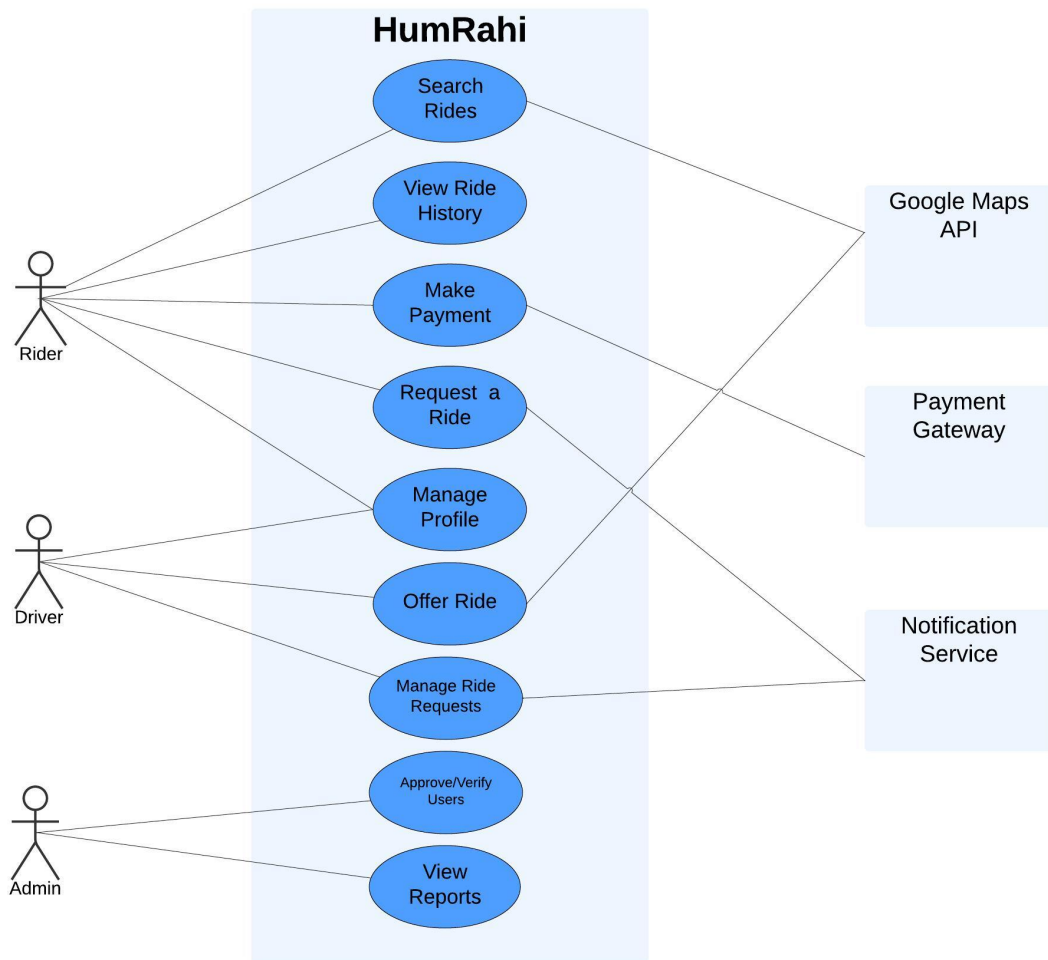


Figure 2.1: Use Case Diagram

2.1.1 Detailed Use Cases

Use Case Name	Search Rides
Scope	Humrahi Application
Level	User goal level
Primary Actor	Rider
Stakeholders and Interest	Riders: Want to find available rides that match their preferences, such as pickup location, destination, and time. Drivers: Want their offered rides to be visible to potential riders. System Admin: Interested in ensuring ride search functionality is efficient and reliable.
Preconditions	The rider is logged into the system. Available rides are stored in the system with updated data (e.g., routes, timing).
Success Guarantee/ Postcondition	The rider is able to view a list of available rides that match their search criteria.
Main Success Scenario	The rider accesses the search rides feature. The rider enters search criteria, such as pickup location, destination, and time. The system retrieves matching rides from the database. The system displays the available rides that match the entered criteria.
Extensions	If no rides are found, the system displays an appropriate message with suggestions for modifying the search criteria. If there is an issue connecting to the Google Maps API, the system notifies the rider of the error.
Special Requirements	The system should allow filtering rides based on additional attributes, such as ride cost, driver rating, and vehicle type. The system should use Google Maps API for route verification.
Technology and Data Variations	Ride data is retrieved from the Humrahi database, while route information is integrated with Google Maps API.

Table 2.1: Detailed Use Case for Search Rides

Use Case Name	Offer Ride
Scope	Humrahi Application
Level	User goal level
Primary Actor	Driver
Stakeholders and Interest	Riders: Interested in finding available rides that meet their schedule and route. Drivers: Want to offer rides to potential riders to share costs and earn extra income. System Admin: Wants to ensure that rides offered are reliable and follow community guidelines.
Preconditions	he driver is logged into the system. The driver has a verified profile and vehicle details are updated in the system.
Success Guarantee/ Postcondition	The driver successfully offers a ride, and the ride is made available to riders searching for it.
Main Success Scenario	The driver accesses the "Offer Ride" feature. The driver provides details about the ride (e.g., route, date, time, available seats, cost). The system verifies the route using Google Maps API and stores the ride details in the database. The system confirms that the ride is successfully offered and notifies the driver.
Extensions	If the ride details are incomplete, the system prompts the driver to provide the missing information. If there is an error in route verification, the system displays an appropriate message and suggests correcting the route.
Special Requirements	The system should validate the route using Google Maps API and estimate travel time and distance. The system should notify the driver when a rider requests to join the ride.
Technology and Data Variations	Ride details are stored in the Humrahi database, and route information is verified with Google Maps API.

Table 2.2: Detailed Use Case for Offer Ride

Use Case Name	Make Payment
Scope	Humrahi Application
Level	User goal level
Primary Actor	Rider
Stakeholders and Interest	Riders: Want to make secure payments for rides booked through the application. Drivers: Want to receive payments securely for the rides they offer. System Admin: Interested in ensuring ride search functionality is efficient and reliable.
Preconditions	The rider has selected a ride to book. The rider has a valid payment method linked to their account.
Success Guarantee/ Postcondition	The payment is processed successfully, and the ride is booked.
Main Success Scenario	The rider selects the "Make Payment" option after choosing a ride. The system connects to the payment gateway. The rider enters payment details and confirms the transaction. The payment gateway processes the payment and sends confirmation to the system. The system confirms the booking to the rider.
Extensions	If the payment is declined, the system notifies the rider and allows them to retry or choose a different payment method. If there is a connection issue with the payment gateway, the system displays an error message and prompts the rider to try again later.
Special Requirements	The payment process must be secure, complying with industry standards such as PCI-DSS. The system should allow the rider to view the payment receipt and transaction details after a successful payment.
Technology and Data Variations	Payment information is processed through an external payment gateway, and confirmation is stored in the Humrahi database.

Table 2.3: Detailed Use Case for Make Payment

Use Case Name	Manage Profile
Scope	Humrahi Application
Level	User goal level
Primary Actor	Rider/Driver
Stakeholders and Interest	Riders/Drivers: Want to update personal details, contact information, and preferences to enhance their experience. System Admin: Wants to ensure all user data is accurate and up-to-date.
Preconditions	The user is logged into the system.
Success Guarantee/ Postcondition	The user profile is successfully updated, and changes are saved in the system.
Main Success Scenario	The user accesses the "Manage Profile" feature. The user updates their information (e.g., name, contact details, vehicle information for drivers). The system validates the updated information. The system saves the changes and confirms the update to the user.
Extensions	If the entered information is invalid, the system prompts the user to correct the errors. If there is a system error while saving the changes, the system displays an error message and asks the user to try again later.
Special Requirements	The system should validate email addresses and phone numbers before saving changes. The system should allow the user to upload or update profile pictures.
Technology and Data Variations	Ride data is retrieved from the Humrahi database, while route information is integrated with Google Maps API.

Table 2.4: Detailed Use Case for Manage Profile

Use Case Name	Request a Ride
Scope	Humrahi Application
Level	User goal level
Primary Actor	Rider
Stakeholders and Interest	Riders: Want to request a ride that matches their schedule and route. Drivers: Want to be informed when a rider requests to join their ride. System Admin: Wants to monitor ride requests to prevent misuse or fraudulent activities.
Preconditions	The rider is logged into the system. The rider has found an available ride that meets their requirements.
Success Guarantee/ Postcondition	The ride request is successfully sent to the driver, and the rider receives confirmation.
Main Success Scenario	The rider selects an available ride from the search results. The rider clicks the "Request a Ride" button. The system sends a notification to the driver, indicating the ride request. The driver reviews the request and either approves or declines it. If approved, the rider is notified, and the ride is confirmed.
Extensions	If the driver declines the ride request, the system notifies the rider, and the rider can search for other available rides. If the rider cancels the request before the driver's response, the system updates the ride status accordingly.
Special Requirements	The system should provide real-time notifications for the driver and rider regarding the status of the request. The system should allow the rider to communicate with the driver via in-app messaging once the ride is confirmed.
Technology and Data Variations	Notification data is sent using the Notification Service, and the ride request is stored in the Humrahi database.

Table 2.5: Detailed Use Case for Request a Ride

Use Case Name	View Ride History
Scope	Humrahi Application
Level	User goal level
Primary Actor	Rider
Stakeholders and Interest	Riders: Want to view details of past rides, including payment details, ride duration, and driver information. Drivers: Want to review their completed rides, including rider feedback and payment details. System Admin: Interested in monitoring ride history for audit purposes and to resolve disputes.
Preconditions	The user (rider or driver) is logged into the system. The system has stored records of the user's past rides.
Success Guarantee/ Postcondition	The user is able to view the complete history of their past rides.
Main Success Scenario	The user accesses the "View Ride History" feature. The system retrieves all past ride records associated with the user. The system displays the ride history, including information such as ride date, route, driver/rider details, and payment.
Extensions	If there are no past rides available, the system displays an appropriate message. If the user wishes to view more details about a specific ride, the system allows them to click on that ride to view additional information (e.g., driver/rider ratings, route details).
Special Requirements	The system should allow users to filter ride history based on time periods (e.g., last week, last month) or ride status (e.g., completed, canceled). The system should provide a summary of total rides and payments for a specified time period.
Technology and Data Variations	Ride history data is retrieved from the Humrahi database, and additional payment details are integrated from the payment gateway logs.

Table 2.6: Detailed Use Case for View Ride History

Use Case Name	Approve/Verify Users
Scope	Humrahi Application
Level	User goal level
Primary Actor	Admin
Stakeholders and Interest	Admin: Wants to verify users' identity and approve new accounts to maintain security and trust in the system. Users: Want their accounts to be verified quickly so they can start using the application.
Preconditions	New user accounts have been created and are pending verification.
Success Guarantee/ Postcondition	The user is successfully verified and approved to use the application.
Main Success Scenario	The admin accesses the "Approve/Verify Users" feature. The system displays a list of pending user accounts. The admin reviews the details provided by each user (e.g., ID verification, contact details). The admin approves or rejects the user account. The system notifies the user of the admin's decision.
Extensions	If the driver does not respond to the request within a specific timeframe, the system sends a reminder notification. If the ride is canceled by the driver, all pending requests are automatically declined.
Special Requirements	The system should provide a summary of rider details for each request, including ratings and comments from previous drivers. The system should allow drivers to communicate with riders before deciding on a request.
Technology and Data Variations	Ride request data is retrieved from and updated in the Humrahi database, and notifications are sent using the Notification Service.

Table 2.7: Detailed Use Case for Approve/Verify Users

Use Case Name	View Reports
Scope	Humrahi Application
Level	User goal level
Primary Actor	Admin
Stakeholders and Interest	Admin: Wants to monitor system performance, user activities, and other relevant metrics to ensure smooth functioning. Owners: Interested in insights into user behavior and ride statistics to make informed business decisions.
Preconditions	The admin is logged into the system.
Success Guarantee/ Postcondition	The admin successfully views the reports and uses them for analysis and decision-making.
Main Success Scenario	The admin accesses the "View Reports" feature. The system displays different types of reports (e.g., ride statistics, payment summaries, user activity). The admin selects a report type to view. The system retrieves the relevant data and displays it in a structured format.
Extensions	If there is no data available for a specific report type, the system notifies the admin. If there is an issue retrieving the report data, the system displays an error message.
Special Requirements	The reports should include visual elements such as charts and graphs for easier understanding. The system should allow the admin to export the reports as PDFs or spreadsheets.
Technology and Data Variations	Report data is retrieved from the Humrahi database and displayed using the reporting module.

Table 2.8: Detailed Use Case for View Reports

2.2 Functional Requirements

2.2.1 UI/UX and Profile Management

1. The system shall allow users to sign up and log in using email, phone number, or social media accounts.
2. The system shall enable users to verify their identity through email or phone-based verification.
3. The system shall provide users with the ability to edit and manage their profile information, including personal details and preferences.

2.2.2 Ride Matching and Booking

1. The system shall allow drivers to create ride offers by specifying route details, availability, and capacity.
2. The system shall allow passengers to create ride requests by providing their destination and preferred timing.
3. The system shall use an algorithm to match ride offers with ride requests based on proximity, route, and user preferences.

2.2.3 Route Optimization and Navigation

1. The system shall provide route planning and navigation for drivers using integration with mapping services.
2. The system shall provide real-time traffic updates and adjust routes accordingly to avoid congestion.
3. The system shall calculate and display the estimated time of arrival (ETA) and distance for each ride.

2.2.4 Dynamic Pricing Model

1. The system shall calculate the estimated fare for each ride based on distance, demand, and time before ride confirmation.

2. The system shall allow users to make payments within the app using multiple payment options (e.g., credit card, e-wallet).
3. The system shall adjust pricing dynamically during peak times or special conditions (e.g., holidays).

2.2.5 Safety and Security Features

1. The system shall allow real-time ride tracking, enabling users to share their location with others during a ride.
2. The system shall collect and display ratings and reviews for both drivers and passengers after each ride.
3. The system shall verify driver licenses and vehicle information during the sign-up process for drivers.

2.2.6 User Feedback and Support

1. The system shall provide an in-app helpdesk and support options for user queries and issues.
2. The system shall allow users to rate and provide feedback on their ride experience after ride completion.
3. The system shall automatically analyze user reviews and feedback using natural language processing (NLP) to detect sentiment and identify potential issues.

2.2.7 Notifications and Alerts

1. The system shall send push notifications to users regarding ride requests, confirmations, and status updates.
2. The system shall send alerts to users about upcoming rides, cancellations, or delays.
3. The system shall allow users to customize their notification settings based on personal preferences.

2.2.8 User Analytics and Reporting

1. The system shall collect data on user preferences, ride patterns, and feedback to improve the service.

2. The system shall provide an analytics dashboard for administrators to monitor user activities and app performance.
3. The system shall generate reports on environmental impact (e.g., carbon emissions reduced) and cost savings achieved by users.

2.2.9 Real-time Ride Analytics and Processing Module

1. The system shall use Apache Kafka to collect real-time data on ride duration, distance, and route deviations.
2. The system shall provide live analytics on traffic conditions during the ride to adjust routes if necessary.
3. The system shall be scalable to support increased volumes of live data for future expansion of the platform.

2.3 Non-Functional Requirements

2.3.1 Reliability

- The system shall achieve a mean time between failures (MTBF) of at least 1,000 hours.
- In case of a server failure, the system shall provide recovery mechanisms to restore data within 5 minutes to minimize downtime.
- The system shall log all errors, and users shall be notified of a failure within 1 minute of its detection.

2.3.2 Usability

- **USE-1:** The system shall allow a new user to complete the sign-up process in less than 2 minutes.
- **USE-2:** The system shall enable users to book a ride with no more than 5 interactions from the home screen.
- **USE-3:** The system shall comply with Web Content Accessibility Guidelines (WCAG) 2.1 Level AA to ensure accessibility for users with disabilities.

2.3.3 Performance

- **PER-1:** The system shall display search results for available rides within 2 seconds of a user request.
- **PER-2:** The system shall handle up to 1,000 concurrent users without performance degradation.
- **PER-3:** The system shall provide real-time traffic updates with a latency of no more than 5 seconds.

2.3.4 Security

- **SEC-1:** The system shall prevent unauthorized access by requiring user authentication with a minimum password length of 8 characters.
- **SEC-2:** The system shall lock a user account for 30 minutes after 5 consecutive unsuccessful login attempts.
- **SEC-3:** All sensitive data, such as payment information and personal identification details, shall be encrypted during transmission and storage using AES-256 encryption.

2.4 Domain Model

Create a representation of the domain model for your project.

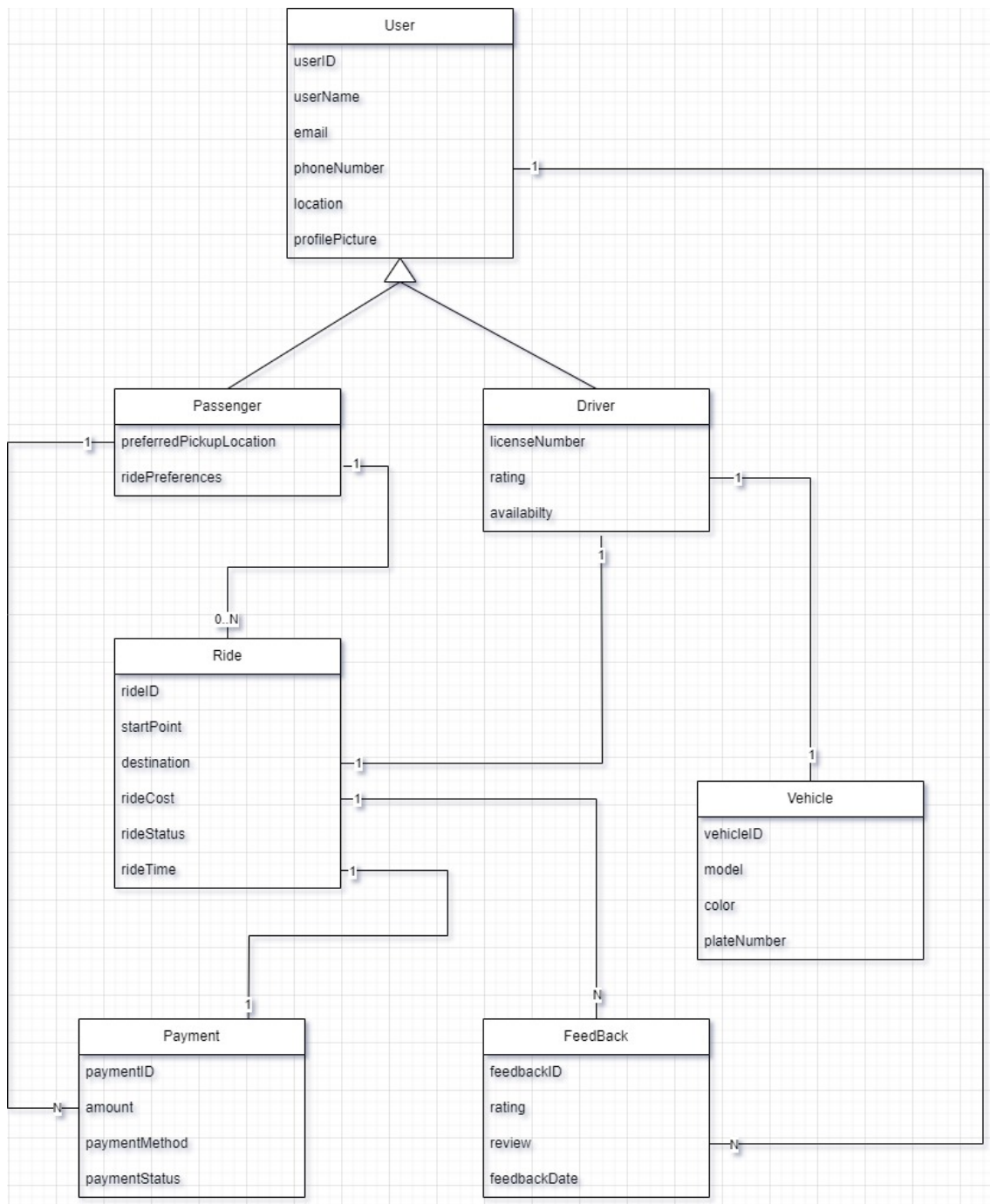


Figure 2.2: Domain Model Diagram of HumRahi

Chapter 3

System Overview

3.1 Architectural Design

For Humrahi, we can structure the system into a 3-layer architecture, each with distinct roles and responsibilities:

3.1.0.1 1. Presentation Layer

Description: The Presentation Layer is the user-facing part of the system, handling all interactions between users (commuters and drivers) and the application. It provides the interface where users can log in, register, search for rides, view route details, and make payments. This layer is built using technologies that support web and mobile responsiveness for a seamless user experience.

3.1.0.2 2. Business Logic Layer

Description: The Business Logic Layer is the heart of the system, where all business logic and decision-making happen. It connects the front end and the database layers, processing user requests and managing the system's core functionalities like ride matching, payment processing, and route optimization. It enforces the rules for how users and data interact.

3.1.0.3 3. Data Layer

The Data Layer manages the storage, retrieval, and updating of all persistent data. It stores information about users, rides, payments, and logs. This layer ensures data integrity, security, and availability to the application layer whenever needed.

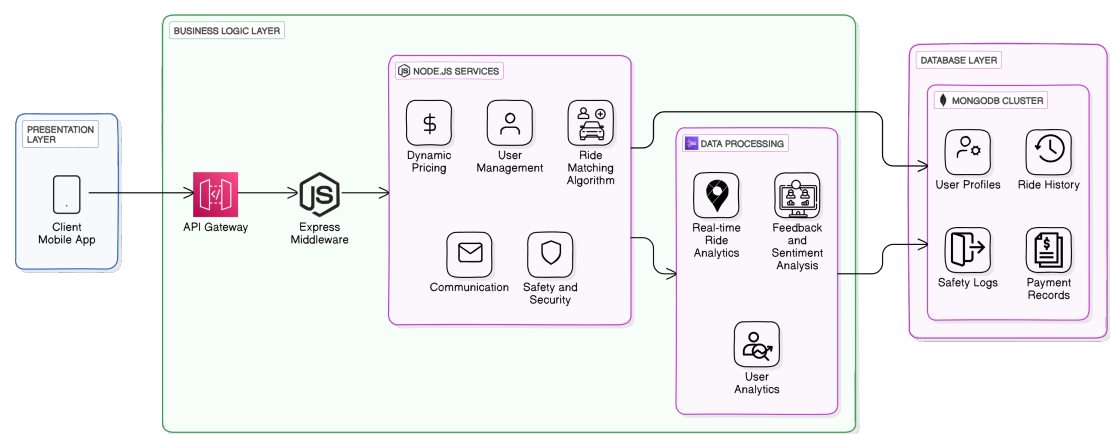


Figure 3.1: Architecture Diagram

3.2 Design Models

3.2.1 Activity Diagram

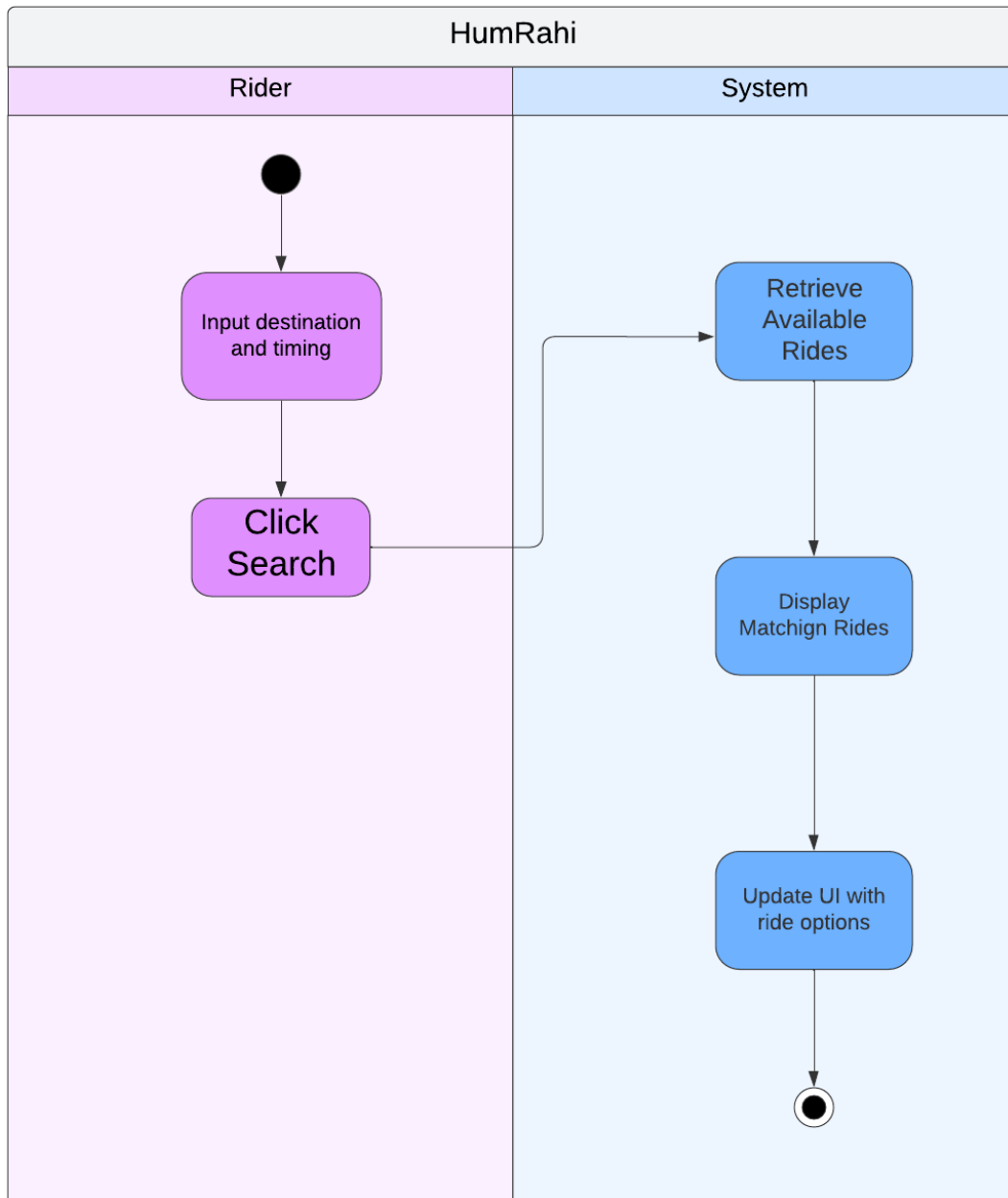


Figure 3.2: Activity Diagram for Search Rides

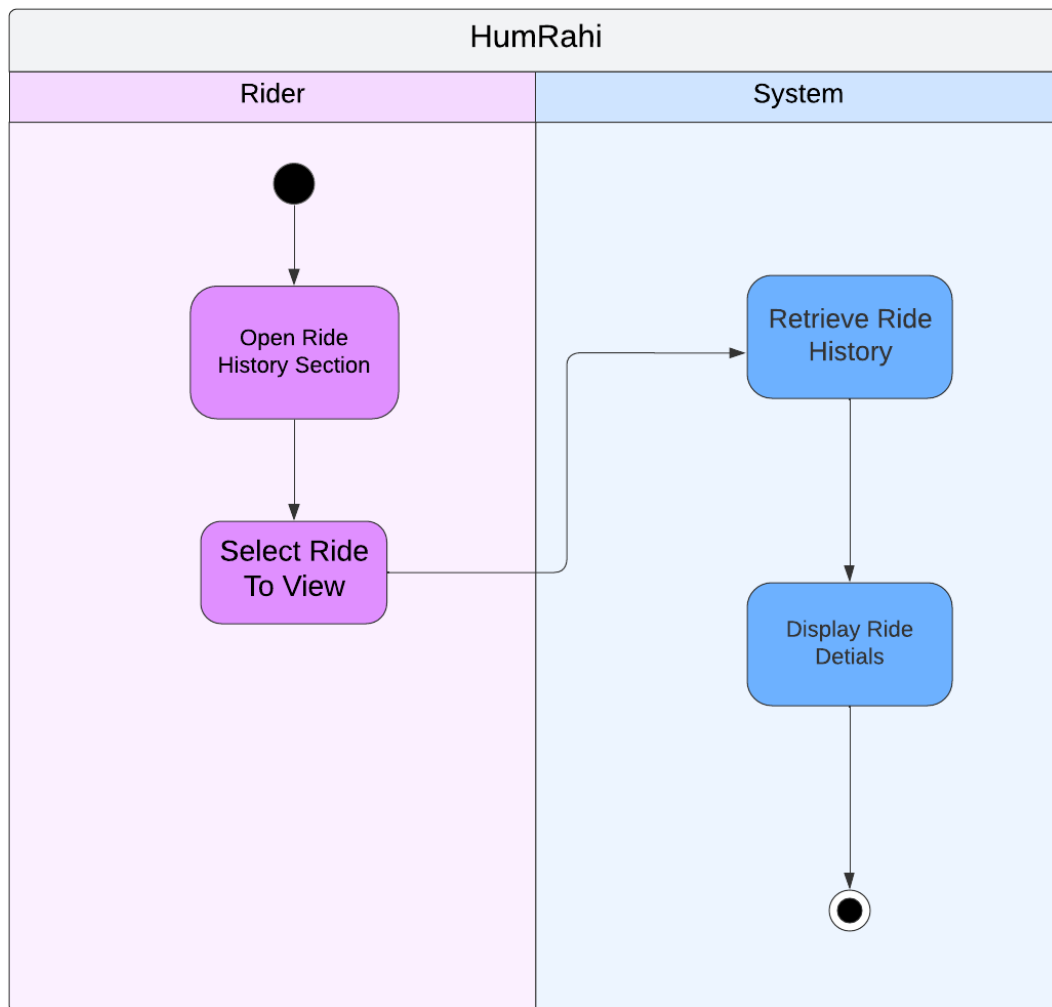


Figure 3.3: Activity Diagram for View Ride History

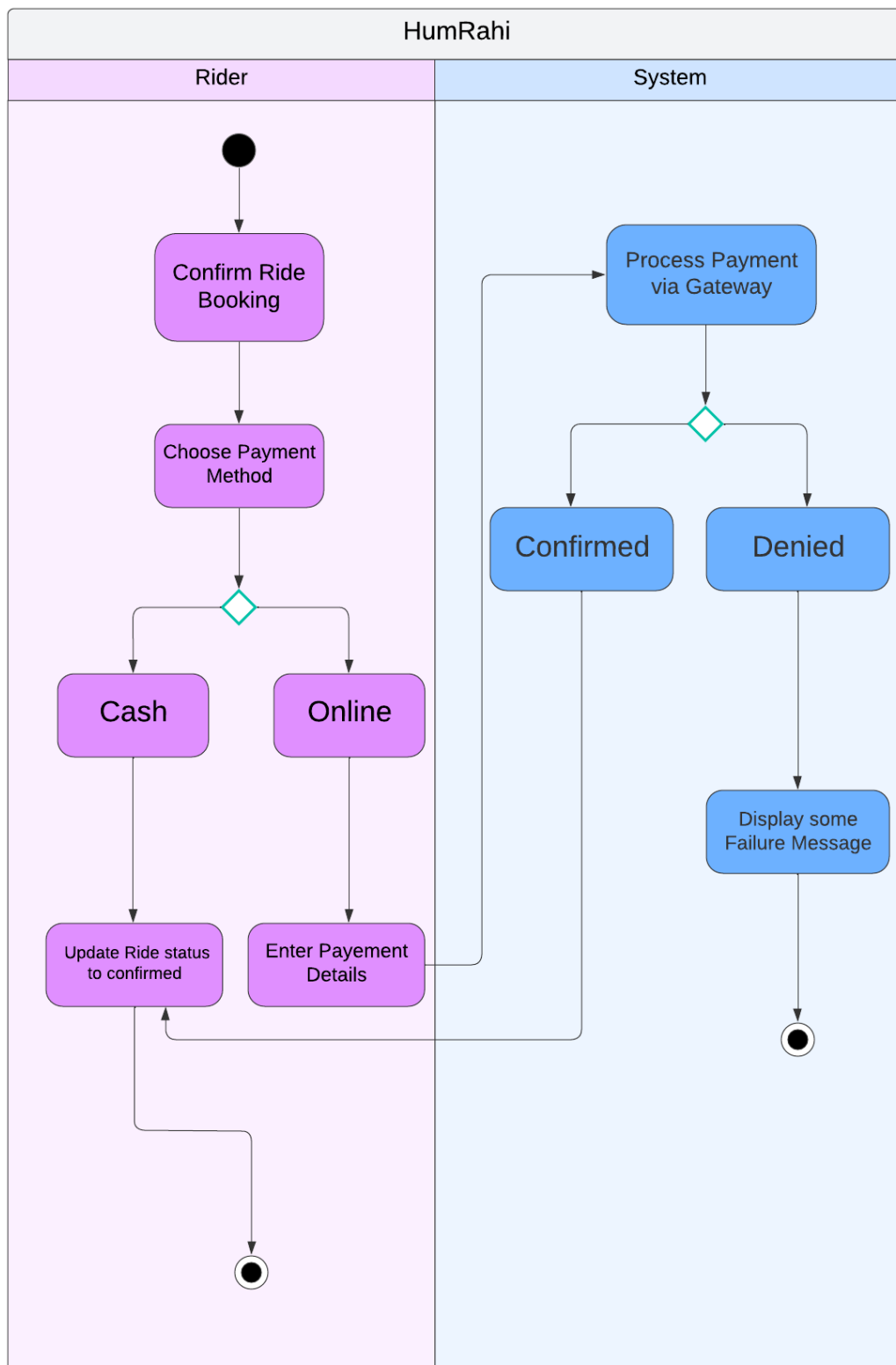


Figure 3.4: Activity Diagram for Make Payment

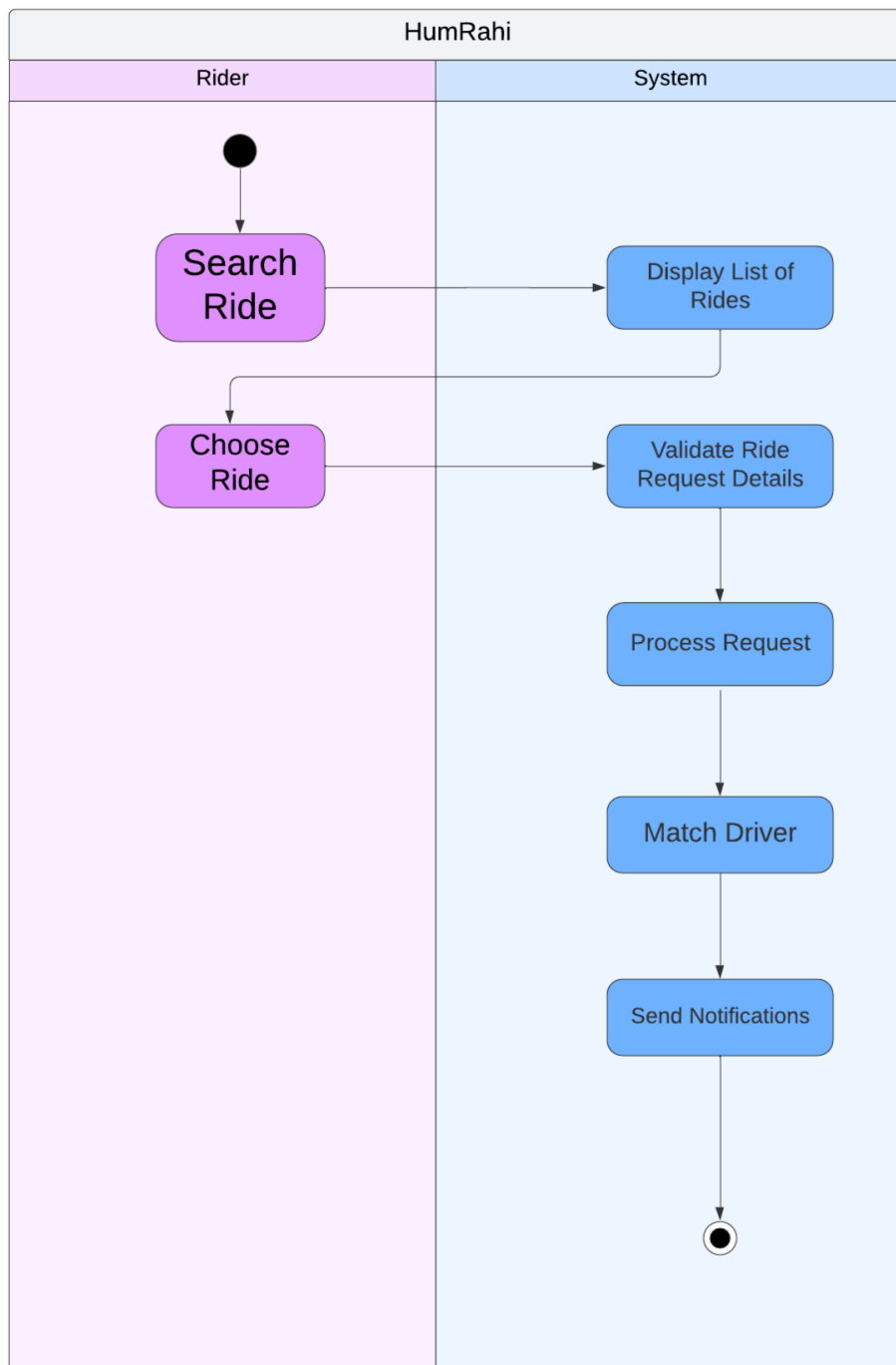


Figure 3.5: Activity Diagram for Requesting a Ride

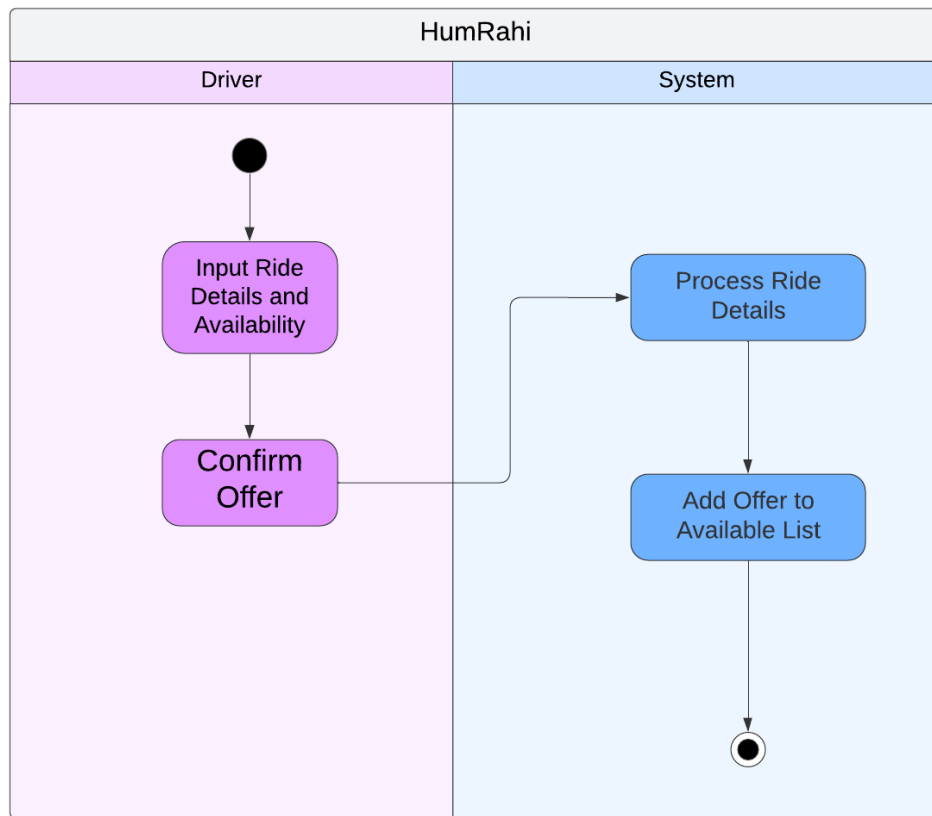


Figure 3.6: Activity Diagram for Offer a Ride

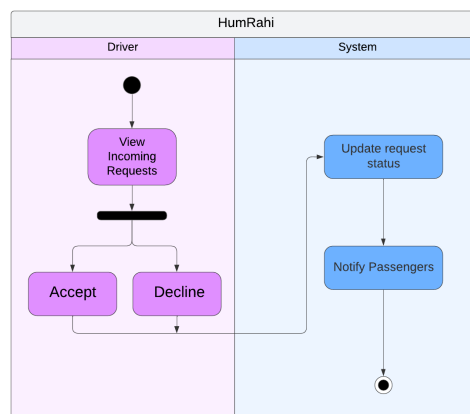


Figure 3.7: Activity Diagram for Manage Ride Requests

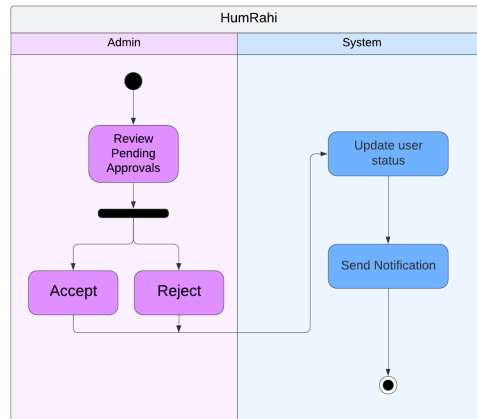


Figure 3.8: Activity Diagram for Approve/Verify Users

3.2.2 Data Flow Diagram

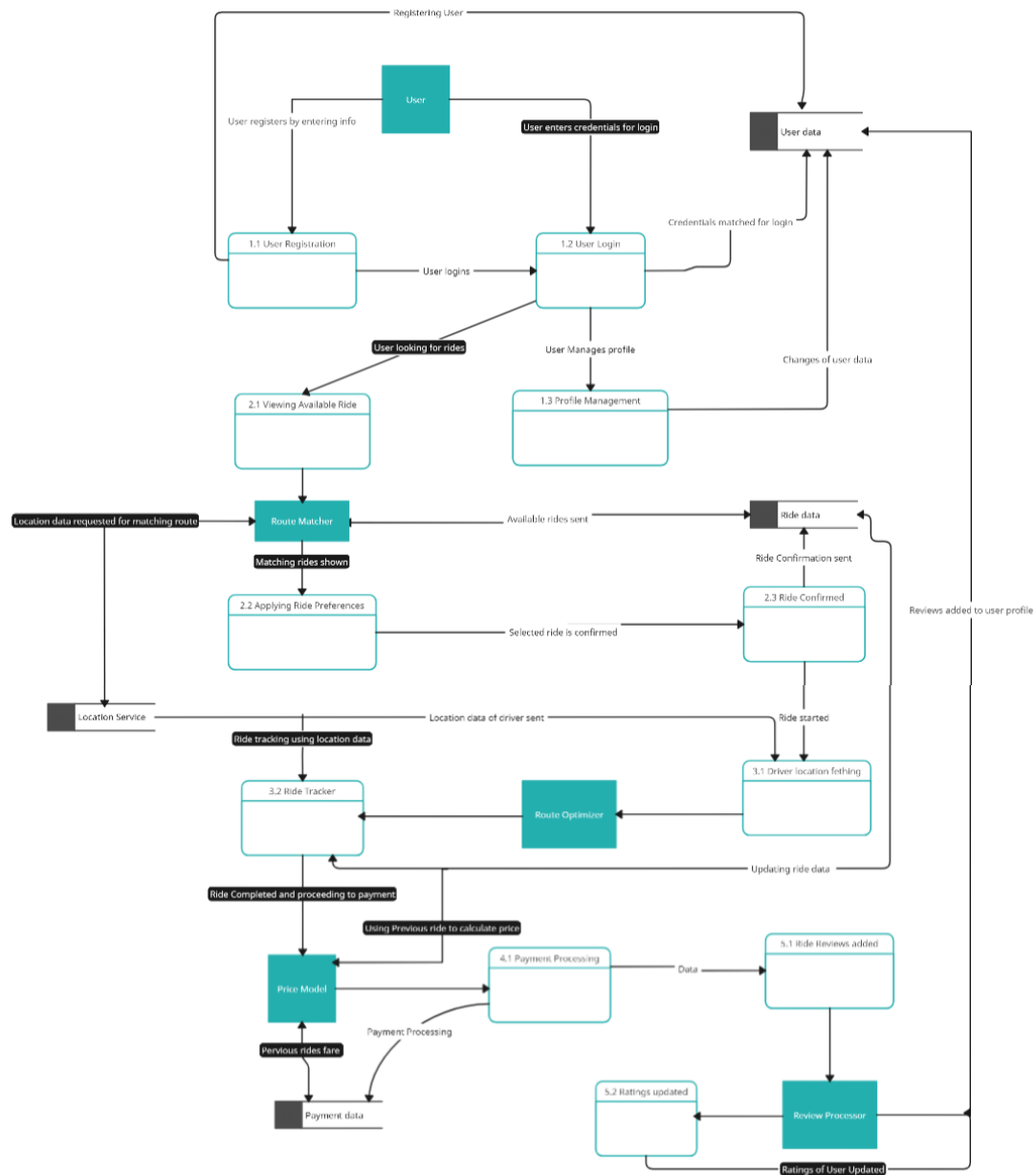


Figure 3.9: Data Flow Diagram

3.2.2.1 1. User Interaction

- **Users:** The end-users of the system include both passengers and drivers. They are responsible for interacting with the system by requesting rides, offering rides, updating profiles, and managing payments.
 - Passenger: Users who request rides.
 - Driver: Users who offer rides through the platform.
- **Profile Management:** Users manage their profiles by providing and updating personal details like name, email, phone number, and in the case of drivers, vehicle information. This data is critical for matching rides and ensuring safety within the system.

3.2.2.2 2. Data Input and Processing

- **Ride Request and Offering:**
 - **Ride Request:** Passengers provide inputs for their ride request, specifying their origin, destination, and time of travel.
 - **Ride Offering:** Drivers provide availability information, including their location, vehicle details, and preferences for ride requests.
- **Ride Matching Process:**
 - **Matching Algorithm:** The system processes ride requests by matching passengers with available drivers based on parameters like origin, destination, and real-time availability. This is done using route-matching algorithms that consider proximity and driver preferences.

3.2.2.3 3. Payment Processing

- **Fare Calculation:** When a ride is completed, the system processes payments by calculating the total fare using the pricing analytics engine. This engine incorporates:
 - Petrol Prices.
 - Recent Prices.
 - Distance traveled.
 - Ride duration.

- **Payment Gateway:** The system securely processes payments between passengers and drivers, ensuring that the transaction details are logged and stored in the database.

3.2.2.4 4. Notification System

- **Ride Confirmation & Updates:**
 - The system sends real-time notifications to both passengers and drivers regarding ride statuses such as `ride matched`, `ride started`, and `ride completed`.
 - These notifications are automatically triggered based on system events and sent to users via in-app notifications, emails, or SMS.
- **Ride Review:** After the ride is completed, both passengers and drivers are prompted to leave a review, helping improve trust and accountability within the system.

3.2.2.5 5. Safety and Security

- **User Authentication:** The system securely authenticates all users using a login system based on email and passwords. Users are verified through email during account creation to ensure security.
- **Driver Verification:** Drivers are subject to additional verification processes, including submission of vehicle information and legal driving credentials. This ensures that only verified drivers can offer rides within the platform.

3.2.2.6 6. Data Storage

- **MongoDB:**
 - All data related to the ride-matching process, user profiles, ride requests, payments, and reviews are stored in a **MongoDB** database. This NoSQL database offers a flexible schema to handle the dynamic and scalable nature of carpooling data.
 - **Entities Stored:**
 - * **User Profiles:** Stores data related to both passengers and drivers.
 - * **Ride Requests:** Data associated with each ride request and its status.
 - * **Matched Rides:** Stores details of completed and ongoing rides, including ride metadata.
 - * **Payment Records:** Stores payment transaction logs and fare details.

- * **Ride Reviews:** Stores ratings and feedback from users.
- **Real-Time Updates:** The system continuously updates MongoDB with new ride requests, matches, payment records, and user reviews to ensure that the data is always current and accessible for both passengers and drivers.

3.2.3 System-Level Sequence Diagram

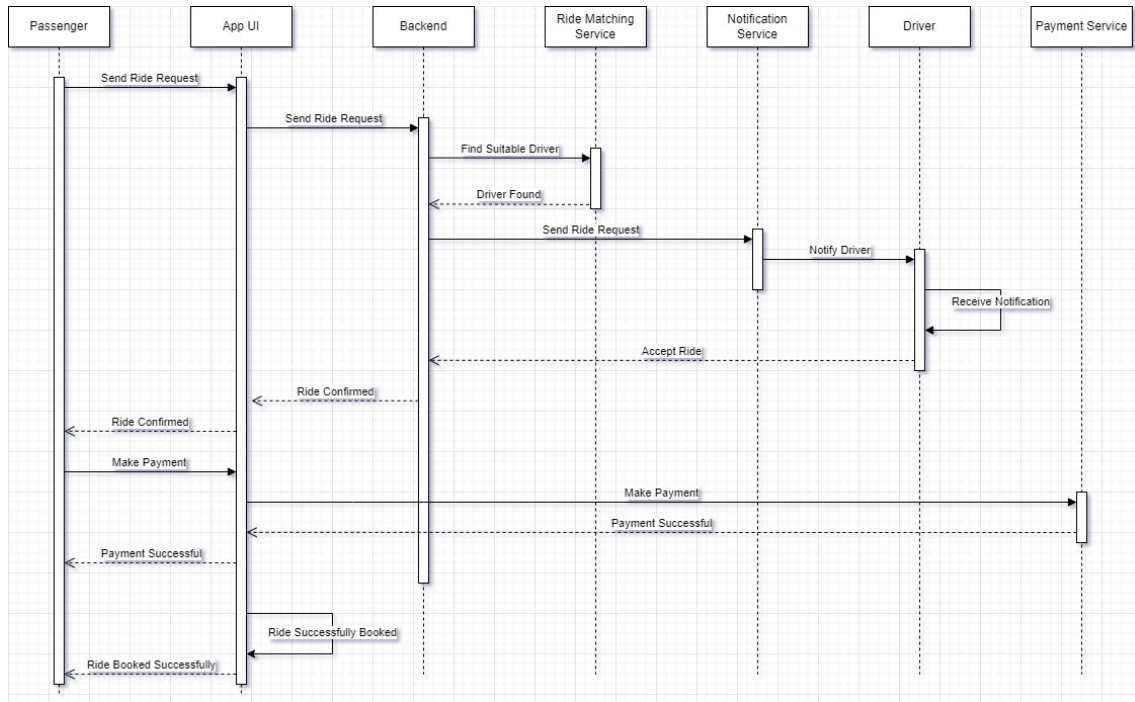


Figure 3.10: System-Level Sequence Diagram

3.3 Data Design

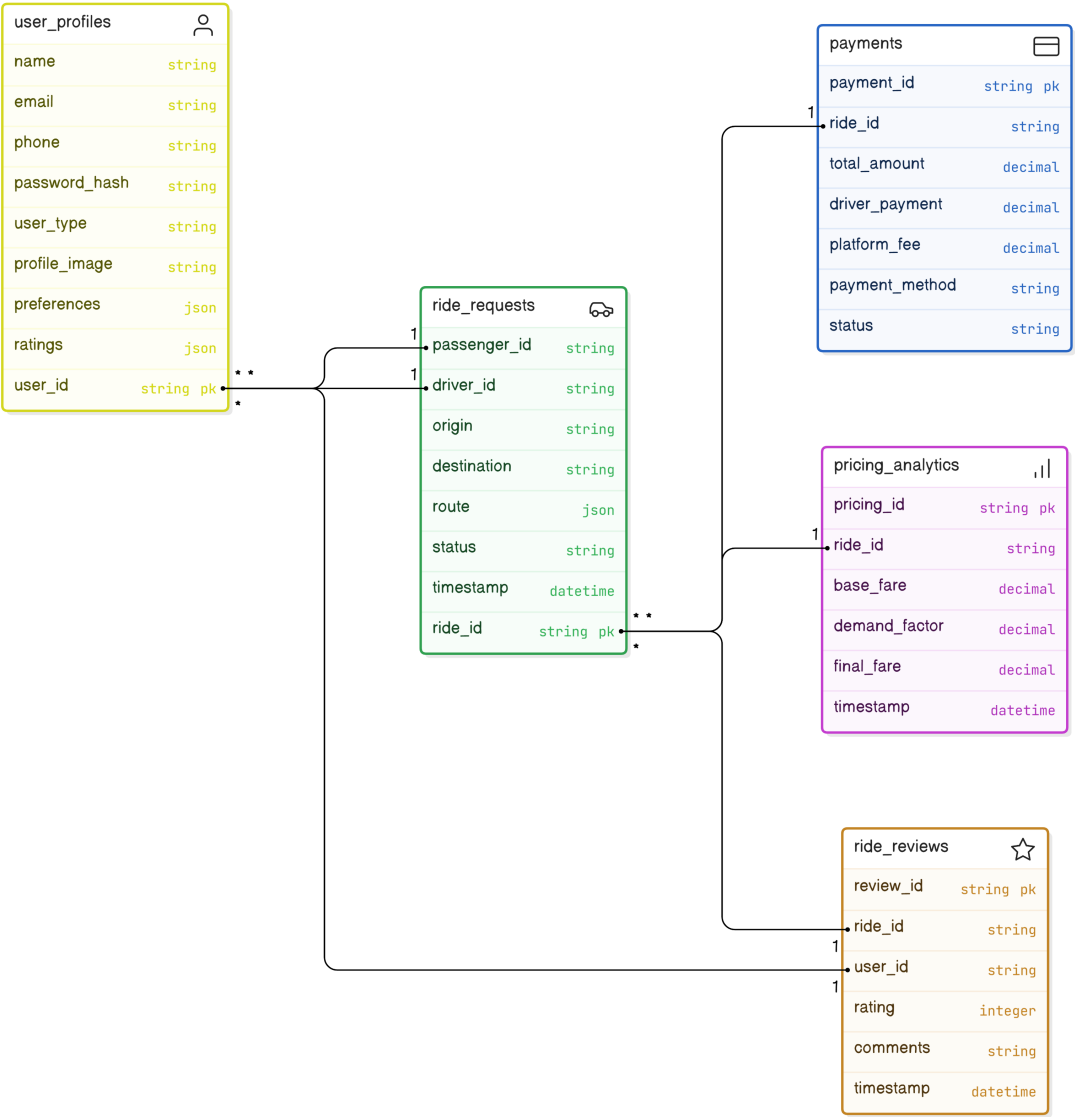


Figure 3.11: Entity Relationship Diagram

3.3.0.1 1. User Profiles

- **Structure:** Each user profile contains the following fields:
 - `user_id`: Unique identifier for each user.
 - `name`: Full name of the user.
 - `email`: User's email address.
 - `phone`: User's phone number.
 - `user_type`: Whether the user is a passenger or a driver.
 - `vehicle_info` (for drivers): Information about the driver's vehicle, including make, model, and license plate.
 - `ratings`: A record of the average user rating.
- **Processing:** User profiles are created during registration. Users can manually update their profile details. The system also processes user ratings based on completed rides, allowing drivers and passengers to rate each other.
- **Storage:** User profiles are stored in MongoDB, which allows for a flexible schema to support the differences between passenger and driver profiles.

3.3.0.2 2. Ride Requests

- **Structure:** Ride request entities store information regarding a passenger's request for a ride:
 - `ride_request_id`: Unique identifier for each ride request.
 - `passenger_id`: Reference to the passenger making the request.
 - `origin`: Pickup location of the ride.
 - `destination`: Drop-off location of the ride.
 - `status`: Current status of the request (e.g., pending, matched, completed).
- **Processing:** The system processes new ride requests, matches them to available drivers, and updates the status as the ride progresses. When matched, the ride request links to a specific driver and matched ride entity.
- **Storage:** Ride requests are stored in MongoDB, with references to both the user and driver involved, making it easy to track the lifecycle of a ride.

3.3.0.3 3. Drivers

- **Structure:** Driver entities represent the drivers who offer rides to passengers:
 - `driver_id`: Unique identifier for each driver.
 - `vehicle_info`: Vehicle details including make, model, and license plate.
 - `ratings`: A history of driver ratings from passengers.
- **Processing:** Drivers are associated with one or more ride requests. Their ratings are dynamically updated after completing rides, providing feedback to other users.
- **Storage:** Driver data is stored in MongoDB, linked to the ride requests and matched rides they are associated with.

3.3.0.4 4. Matched Rides

- **Structure:** This entity represents a ride that has been successfully matched between a passenger and a driver:
 - `ride_id`: Unique identifier for the matched ride.
 - `ride_request_id`: Reference to the originating ride request.
 - `driver_id`: Reference to the driver who accepted the ride request.
 - `status`: Current status of the ride (e.g., ongoing, completed).
- **Processing:** Once a ride request is matched to a driver, this entity is created. The system updates the ride status as it progresses and manages payment transactions associated with the ride.
- **Storage:** Matched rides are stored in MongoDB, with references to both the driver and the passenger.

3.3.0.5 5. Payments

- **Structure:** Payment entities capture the details of transactions for completed rides:
 - `payment_id`: Unique identifier for the payment.
 - `ride_id`: Reference to the matched ride associated with the payment.
 - `total_amount`: Total fare amount calculated for the ride.
 - `status`: Status of the payment (e.g., pending, completed).

- **Processing:** The system calculates the fare based on the distance traveled and dynamic pricing algorithms. It processes payments after the ride is completed, ensuring that both the passenger and driver have completed the trip.
- **Storage:** Payment data is stored in MongoDB, with references to the matched ride and user.

3.3.0.6 6. Ride Reviews

- **Structure:** Ride reviews allow passengers and drivers to provide feedback on each other:
 - `review_id`: Unique identifier for the review.
 - `ride_id`: Reference to the associated ride.
 - `user_id`: Reference to the user who left the review.
 - `rating`: Numeric rating (1-5).
 - `comments`: Optional text field for additional feedback.
- **Processing:** The system enables users to leave reviews after completing a ride. Ratings are aggregated for drivers and passengers to maintain an average score.
- **Storage:** Ride reviews are stored in MongoDB, linked to the user and the matched ride.

3.3.0.7 7. Pricing Analytics

- **Structure:** Pricing analytics are used to dynamically calculate fares based on factors such as distance, time, and demand:
 - `analytics_id`: Unique identifier for the pricing analytics data.
 - `ride_id`: Reference to the associated ride.
 - `fare_components`: Breakdown of various factors contributing to the fare (e.g., base fare, surge pricing, discounts).
- **Processing:** The system uses pricing analytics to adjust fares in real-time, taking into account route, demand, and ride conditions.
- **Storage:** Pricing analytics are stored in MongoDB, associated with matched rides for fare calculation and auditing.

3.3.0.8 Data Storage Items:

MongoDB: The primary database used for all entities. MongoDB's flexible NoSQL schema supports dynamic data storage for the diverse data structures in the carpooling system, including user profiles, ride requests, matched rides, payments, reviews, and pricing analytics.

Bibliography